

NumPy- Array Operations

NumPy provides a wide range of operations that can perform on arrays, including.

- ✓ Arithmetic operations.
- ✓ Scalar Operations.
- ✓ Statistical Operation.
- ✓ Conditional Operation.

NumPy - Arithmetic Operations.

NumPy's arithmetic operations are widely used due to their ability to perform simple and efficient calculations on arrays.

NumPy is a popular library in Python for numerical computing, provides a variety of arithmetic operations that can be performed on arrays. These operations include addition, subtraction, multiplication, division, and more. Below are some common arithmetic operations in NumPy:

Element-wise Operation	Operator	Function
Addition	+	<code>add()</code>
Subtraction	-	<code>subtract()</code>
Multiplication	*	<code>multiply()</code>
Division	/	<code>divide()</code>
Exponentiation	**	<code>power()</code>
Modulus	%	<code>mod()</code>

Addition:-

we can use the both `+` operator and the built-in function `add()` to perform element-wise addition between two NumPy arrays.

```
import numpy as np
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
z1=x+x
z2=y+y
p1=np.add(x,x)
```

```
p2=np.add(y,y)
print(z1)
print(z2)
print(p1)
print(p2)
```

Subtraction:-

In NumPy, we can either use the - operator or the subtract() function to perform element-wise subtraction between two NumPy arrays.

```
import numpy as np
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
z1=x-x
z2=y-y
p1=np.subtract(x,x)
p2=np.subtract(y,y)
print(z1)
print(z2)
print(p1)
print(p2)
```

Multiplication:-

For element-wise multiplication, we can use the * operator or the multiply() function

```
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
z1=x*x
z2=y*y
p1=np.multiply(x,x)
p2=np.multiply(y,y)
print(z1)
print(z2)
print(p1)
print(p2)
```

Division (/ or float division):-

We can use either the / operator or the divide() function to perform element-wise division between two numpy arrays.

```
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
z1=x/x
z2=y/y
```

```
p1=np.divide(x,x)
p2=np.divide(y,y)
print(z1)
print(z2)
print(p1)
print(p2)
```

Division (/ or int division):-

We can use either the / operator or the divide() function to perform element-wise division between two numpy arrays.

```
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
z1=x/x
z2=y/y
p1=np.divide(x,x)
p2=np.divide(y,y)
print(z1)
print(z2)
print(p1)
print(p2)
```

Exponentiation (Power):-

Array exponentiation refers to raising each element of an array to a given power.

In NumPy, we can use either the ** operator or the power() function to perform the element-wise exponentiation operation.

```
import numpy as np
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
z1=x**x
z2=y**y
p1=np.power(x,x)
p2=np.power(y,y)
print(z1)
print(z2)
print(p1)
print(p2)
```

Modulus:-

We can perform a modulus operation in NumPy arrays using the % operator or the mod() function.

This operation calculates the remainder of element-wise division between two arrays.

```

import numpy as np
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
z1=x%x
z2=y%y
p1=np.mod(x,x)
p2=np.mod(y,y)
print(z1)
print(z2)
print(p1)
print(p2)

```

NumPy - Scalar Operations.

NumPy, scalar operations refer to arithmetic operations performed between a NumPy array and a scalar value (a single number). These operations are applied element-wise, meaning that the operation is performed between each element of the array and the scalar value.

Scalar operations in NumPy allow you to efficiently apply arithmetic operations between an array and a scalar value. These operations are performed element-wise, which means the scalar operation is applied to each element of the array individually. This is very useful for manipulating arrays without needing explicit loops.

```

import numpy as np
x=np.arange(1,11)
y=np.arange(1,21).reshape(4,5)
print("=====(+)======")
print(x+10)#Add 1D Array
print(y+10)#Add 2D Array
print("=====-(=)=====")
print(x-10)#Sub 1D Array
print(y-10)#Sub 2D Array
print("=====(*)======")
print(x*5)#Mul 1D Array
print(y*5)#Mul 2D Array
print("=====(/)======")
print(x/2)#Div 1D Array
print(y/2)#Div 2D Array
print("=====(/)======")
print(x//2)#DD 1D Array
print(y//2)#DD 2D Array
print("=====(%)======")
print(x%2)#Modulus 1D Array
print(y%2)#Modulus 2D Array
print("=====(* *)=====")
print(x**2)#Power 1D Array
print(y**2)#Power 2D Array

```

NumPy-Statistical Operation:-

NumPy provides a wide range of statistical operations to compute descriptive statistics on arrays. These operations include calculating measures of central tendency, dispersion, and other statistical metrics. Below are some common statistical operations in NumPy

1) Mean :- `numpy.mean()`

Arithmetic mean is the sum of elements along an axis divided by the number of elements. The `numpy.mean()` function returns the arithmetic mean of elements in the array. If the axis is mentioned, it is calculated along it.

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b = np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
ma=np.mean(a,dtype=int) # mean of a 1D
mb=np.mean(b) # mean of b all elements of 2D array
mbh=np.mean(b,axis=0,dtype=int)
mbv= np.mean(b,axis=1)
print("Mean of 1D:- ",ma)
print("Mean of 2D:- ",mb)
print("Mean of 2D Horizontal",mbh)
print("Mean of 2D Vertical",mbv)
```

2) Median:- `numpy.median()`

Median is defined as the value separating the higher half of a data sample from the lower half. The `numpy.median()`.

Note:- The `dtype` parameter is not directly available in the `np.median()`

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
ma=np.median(a)
mb=np.median(b)
mbh=np.median(b,axis=1)
mbv= np.median(b,axis=0)
print("Median of 1D:- ",ma)
print("Median of 2D:- ",mb)
print("Median of 2D Horizontal",mbh)
print("Median of 2D Vertical ",mbv)
#To change Data Type
print(np.array(mbv, dtype=int))
```

3) Variance:- `numpy.var()`

Variance is a statistical measure that quantifies the spread or dispersion of a set of data points. It indicates how much the individual data points in a dataset differ from the mean (average) of the dataset. A higher variance indicates that the data points are more spread out from the mean, while a lower variance indicates that they are closer to the mean.

Types of Variance:-

Population Variance (σ^2):- Population variance is the variance calculated when considering every individual or data point in the entire population.(use divide N).

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
pva=np.var(a)
pvb=np.var(b)
pvbh=np.var(b,axis=1)
pvbv= np.var(b,axis=0,dtype=int)
print("Population Variance of 1D:- ",pva)
print("Population Variance of 2D:- ",pvb)
print("Population Variance of 2D Horizontal",pvbh)
print("Population Variance of 2D Vertical ",pvbv)
```

Sample Variance (S^2):-

Sample variance is the variance calculated from a subset (sample) of the population. It is used as an estimate of the population variance. Since the sample is only a part of the population, an adjustment (using $n-1$ instead of N).

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
sva=np.var(a,ddof=1)
svb=np.var(b,ddof=1)
svbh=np.var(b,axis=1,ddof=1)
svbv= np.var(b,axis=0,dtype=int,ddof=1)
print("Sample Variance of 1D:- ",pva)
print("Sample Variance of 2D:- ",pvb)
print("Sample Variance of 2D Horizontal",svbh)
print("Sample Variance of 2D Vertical ",svbv)
```

4) Standard Deviation:- `numpy.std()`

Standard deviation is a statistical measure that quantifies the amount of variation or dispersion in a set of data points. It provides an indication of how much individual data points deviate from the mean (average) of the dataset.

Note:- The `dtype` parameter is not directly available in the `np.std()`

Types of standard Deviation:-

Population Standard Deviation (σ):- This is the standard deviation calculated when considering every individual or data point in the entire population.

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
pstda=np.std(a)
pstdb=np.std(b)
pstdbh=np.std(b,axis=1)
pstdbv= np.std(b,axis=0)
print("Population Standard Deviation of 1D:- ",pstda)
print("Population Standard Deviation of 2D:- ",pstdb)
print("Population Standard Deviation of 2D Horizontal",pstdbh)
print("Population Standard Deviation of 2D Vertical ",pstdbv)
```

Sample Standard Deviation (s):- This is the standard deviation calculated from a subset (sample) of the population. It is an unbiased estimator of the population standard deviation.

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
sstda=np.std(a,ddof=1)
sstdb=np.std(b,ddof=1)
sstdbh=np.std(b,axis=1,ddof=1)
sstdbv= np.std(b,axis=0,ddof=1)
print("Sample Standard Deviation of 1D:- ",sstda)
print("Sample Standard Deviation of 2D:- ",sstdb)
print("Sample Standard Deviation of 2D Horizontal",sstdbh)
print("Sample Standard Deviation of 2D Vertical ",sstdbv)
```

Note:- For sample variance or sample standard deviation use (`ddof=1`) as a arguments. `ddof` (Delta Degrees of Freedom)

5) Sum:- **numpy.sum()**

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
sa=np.sum(a)
sb=np.sum(b)
sbh=np.sum(b,axis=1)
sbv= np.sum(b,axis=0)
print("Sum of 1D:- ",sa)
print("Sum of 2D:- ",sb)
print("Sum of 2D Horizontal",sbh)
print("Sum of 2D Vertical ",sbv)
```

6) Minimum and Maximum:

numpy.max(), numpy.min()

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
ma=np.max(a)
mb=np.max(b)
mbh=np.max(b,axis=1)
mbv= np.max(b,axis=0)
print("Maximum Value form 1D:- ",ma)
print("Maximum Value form 2D:- ",mb)
print("Maximum Value form 2D Horizontal",mbh)
print("Maximum Value form 2D Vertical ",mbv)
```

numpy.amax() and numpy.amin()

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
ma=np.amax(a)
mb=np.amax(b)
mbh=np.amax(b,1)
mbv= np.amax(b,0)
print("Maximum Value form 1D:- ",ma)
print("Maximum Value form 2D:- ",mb)
print("Maximum Value form 2D Horizontal",mbh)
print("Maximum Value form 2D Vertical ",mbv)
```


7) Percentile:- `numpy.percentile()`

Percentile (or a centile) is a measure used in statistics indicating the value below which a given percentage of observations in a group of observations fall. The function **`numpy.percentile()`** takes the following arguments.

`numpy.percentile(a, b, axis)`

a:- Input array

b:- The percentile to compute must be between 0-100

axis:- The axis along which the percentile is to be calculated

```
import numpy as np
a = np.array([1,4,3,5,6,7,3,7,6,8,9])
b=np.array([[2,3,1,4,5],[10,12,14,18,11],[1,9,2,8,3]])
print(a)
print(b)
pa=np.percentile(a,50)
pb=np.percentile(b,50)
pbh=np.percentile(b,50,1)
pbv= np.percentile(b,50,0)
print("Percentile Of 1D:- ",pa)
print("Percentile Of 2D:- ",pb)
print("Percentile Of 2D Horizontal",pbh)
print("Percentile Of 2D Vertical ",pbv)
```

NumPy Comparison/Logical Operations:-

NumPy provides several comparison and logical operations that can be performed on NumPy arrays.

NumPy's comparison operators allow for element-wise comparison of two arrays.

NumPy Comparison Operators:-

Operators	Descriptions
< (less than)	returns True if element of the first array is less than the second one
<= (less than or equal to)	returns True if element of the first array is less than or equal to the second one
> (greater than)	returns True if element of the first array is greater than the second one
>= (greater than or equal to)	returns True if element of the first array is greater than or equal to the second one

<code>==</code> (equal to)	returns True if the element of the first array is equal to the second one
<code>!=</code> (not equal to)	returns True if the element of the first array is not equal to the second one

```
import numpy as np
array1 = np.array([1, 2, 3])
array2 = np.array([3, 2, 1])
# less than operator
result1 = array1 < array2
print("array1 < array2:",result1)  # Output: [ True False False]
# greater than operator
result2 = array1 > array2
print("array1 > array2:",result2)  # Output: [False False  True]
# equal to operator
result3 = array1 == array2
print("array1 == array2:",result3)  # Output: [False  True False]
```

NumPy Comparison Functions:-

NumPy also provides built-in functions to perform all the comparison operations.

For example, the `less()` function returns True if each element of the first array is less than the corresponding element in the second array.

Functions	Descriptions
<code>less()</code>	returns element-wise <code>True</code> if the first value is less than the second
<code>less_equal()</code>	returns element-wise <code>True</code> if the first value is less than or equal to second
<code>greater()</code>	returns element-wise <code>True</code> if the first value is greater then second
<code>greater_equal()</code>	returns element-wise <code>True</code> if the first value is greater than or equal to second
<code>equal()</code>	returns element-wise <code>True</code> if two values are equal
<code>not_equal()</code>	returns element-wise <code>True</code> if two values are not equal

```
import numpy as np
array1 = np.array([9, 12, 21])
array2 = np.array([21, 12, 9])
# use of less()
```

```

result = np.less(array1, array2)
print("Using less():",result)  # Output: [ True False False]
# use of less_equal()
result = np.less_equal(array1, array2)
print("Using less_equal():",result)  # Output: [ True  True False]
# use of greater()
result = np.greater(array1, array2)
print("Using greater():",result)  # Output: [False False  True]
# use of greater_equal()
result = np.greater_equal(array1, array2)
print("Using greater_equal():",result)  # Output: [False  True  True]
# use of equal()
result = np.equal(array1, array2)
print("Using equal():",result)  # Output: [False  True False]
# use of not_equal()
result = np.not_equal(array1, array2)
print("Using not_equal():",result)  # Output: [ True False  True]

```

Boolean Indexing:-

Boolean indexing allows you to filter and manipulate array elements based on conditions.

```

import numpy as np
arr = np.array([1, 2, 3, 4, 5])
# Use boolean indexing to filter elements
filter_arr = arr[arr > 2] # array([3, 4, 5])
# Modify elements based on a condition
arr[arr > 2] = 10 # array([ 1,  2, 10, 10, 10])
print(filter_arr)
print(arr)

```

Using np.where():-

The np.where function is useful for element-wise selection based on a condition. It works similarly to the ternary operator in other languages.

```

import numpy as np
# Create an array
arr = np.array([1, 2, 3, 4, 5])
# Use np.where to apply a conditional operation
result = np.where(arr > 2, arr * 2, arr - 2) # array([-1,  0,  6,  8, 10])
print(result)

```

Replacing Elements Based on Condition:-

where we replace elements of an array based on a condition:-

```

import numpy as np
# Create a 2D array

```

```

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# Replace elements greater than 5 with 0
arr[arr > 5] = 0 # array([[1, 2, 3], [4, 5, 0], [0, 0, 0]])
# Use np.where to replace elements based on a condition
result = np.where(arr % 2 == 0, arr, -1) # array([[ -1,  2, -1], [ 4, -1,  0], [ 0,  0,  0]])

```

Broadcasting in NumPy:-

Broadcasting in NumPy is a powerful mechanism that allows NumPy to perform arithmetic operations on arrays of different shapes. It works by virtually expanding the smaller array across the larger array so that they have compatible shapes for element-wise operations. This eliminates the need to create explicitly larger arrays, saving both memory and computational resources.

Rules of Broadcasting:-

- ✓ Arrays are compatible for broadcasting if, for each dimension, the sizes either match or one of them is 1.
- ✓ The arrays are broadcast together to create a new shape, which is the maximum size along each dimension of the input shapes.

```

import numpy as np # possible
# 1D and 2D array
arr1 = np.array([1, 2, 3])
arr2 = np.array([[10], [20], [30]])
result = arr1 + arr2
print(result)

```

```

import numpy as np # possible
# 2D and 3D array
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
arr2 = np.array([[10], [20]])
print(arr1.shape)
print(arr2.shape)
result = arr1 + arr2
print(result)

```

```

import numpy as np # Not possible Why?
# 2D and 3D array
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
arr2 = np.array([[10], [20]])
result = arr1 + arr2
print(result)

```

Assignment

- 1) Write a Python program to find the maximum and minimum value of a given flattened array.
- 2) Write a NumPy program to get the minimum and maximum value of a given array along the axis wise.
- 3) Write a NumPy program to calculate the difference between the maximum and the minimum values of a given array
- 4) Write a NumPy program to compute the 80th percentile for all elements in a given array
- 5) Write a NumPy program to compute the median of given array
- 6) Write a NumPy program to compute the mean, standard deviation, and variance of a given array
- 7) Write a Python program to count number of occurrences of each value in a given array
- 8) Write a NumPy program to add, subtract, multiply, divide arguments element-wise of two array.
- 9) Write a NumPy program to get the powers of an array values element wise.
- 10) Write a NumPy program to calculate the absolute value element-wise.

~:THE END::~~